

Solar Panel Light Tracking

ESET 462/662 Final Course Project

Nikita Choudhary
ESET
Texas A&M University
College Station, TX
nikitachoudhary@tamu.edu

Alvin Anh Nguyen
MXET
Texas A&M University
College Station, TX
anhng275@tamu.edu

Erin Outen
MXET
Texas A&M University
College Station, TX
eouten@tamu.edu

David Parker
MXET
Texas A&M University
College Station, TX
ldmc25@tamu.edu

Bianca Youlton
MXET
Texas A&M University
College Station, TX
b.youl2025@tamu.edu

Abstract— Solar panels use sunlight to generate electricity. For solar panels to reach maximum efficiency, they need to constantly face the sun. This project aims to mimic a large-scale solar tracking system that uses motorized solar panels to follow the sun and maximize energy output. The following describes a small light-tracking system that is made to maximize the amount of light received using a set of photosensors that are attached to a servo motor. The motor's purpose is to rotate and tilt the photosensor to track the light from a flashlight.

The system consists of the Arduino, which processes inputs provided by the photosensors that determine the light intensity from various directions. This relays the data to the Arduino and calculates the direction of the highest light intensity. Then it will send commands to the motor to adjust the orientation of the light panel for maximum exposure. The arc on the sun, when projected perpendicular to the ground, is a straight line, so a single motor is sufficient to follow the sensor along this line.

The project includes feedback control and real-time tracking. Oscillation, jerkiness, and uncontrolled movement will be expected in this project due to inconsistent power transfer from the Arduino to the motor. To solve this problem, we will implement a PID controller to take the error difference between the desired input and the feedback output to smooth the movement of the motor and allow real-time tracking of the light source.

Keywords—PID controller, photosensor, solar panel, feedback

I. INTRODUCTION

One of the main limiting factors that determines the efficiency of solar panels is their exposure to sunlight. Solar panels are often deemed a poor means of energy production because they rely so heavily on factors outside of the user's control. In order to take full advantage of a solar panel's limited access to sunlight, it must be angled in the most optimal position possible. This position, however, changes throughout the year. For example, the ideal angle for residential solar panels in Los Angeles ranges from 19° in the summer to 49° in the winter [1]. Several companies have developed solutions to this problem, such as Array Technologies, which supplies a system of motors that tilt throughout the day to follow the sun [2]. However, according to the SunRun publication, solar trackers are not normalized in the industry due to their high cost; adding these systems can often double the total cost [1].

Furthermore, there is not a diverse supply of solar trackers for applications with individual panels, and these solutions do not exist for residential panels. The product presented in this report provides a model for an inexpensive, lightweight solution to these concerns that will be beneficial in a variety of applications. This project will consist of two photosensors that will determine the position of a platform (hypothetically containing a solar panel) that will continuously and accurately follow the position of the sun or other light source using a single DC motor. The following problem statement will be utilized to direct the development of this system: the solar tracker must use PID control to continuously orient a platform so that it faces the sun.

II. HARDWARE SYSTEM DETAILS

This system consists of an Arduino Nano ESP32, a servo motor, and a long platform that has a resistor and a photoresistor on each end. A voltage of 5V is provided across the system, and the resistance value for each resistor is 100Ω. The resistance value of the photoresistor, as it is adjusted based on the light it detects, is our input to our system and microcontroller. The photoresistors are used to determine the direction of the light, in order to correct a specified error. If more light is detected on the left, the servo motor will move the platform to the left. Similarly, if more light is found on the right, the servo motor will move the platform to the right. The difference between the two measured resistance values of the photoresistors is calculated, and the Arduino's program determines which sensor is detecting more light and instructs the servo motor to move accordingly.

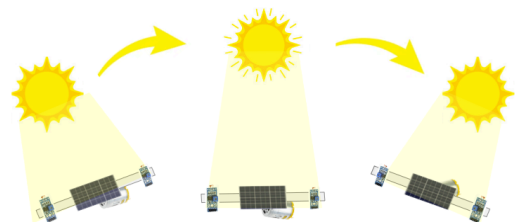


Figure 1: Conceptual Diagram

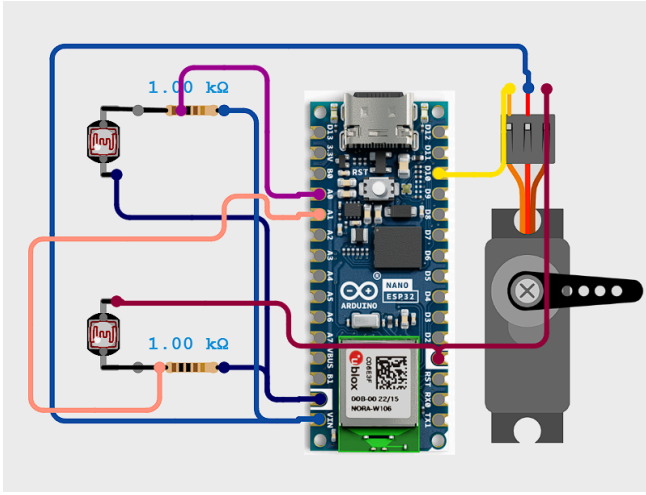


Figure 2: System Block Diagram

III. MODELING SYSTEM DETAILS

The process to obtain a model system of the light tracker contains steps such as deriving the transfer function (s-domain), deriving the discrete transfer function (z-domain), verifying the models, and finally creating a simulated PID control system using MATLAB and Simulink. The system gain of a PID system is demonstrated in Equation 1

$$\text{System Gain} = k_p + \frac{k_i}{s} + k_d s \quad (1)$$

First, the transfer function is determined by using the amount of voltage being transferred into the motor's PWM, which helps rotate the motor. This will be the error for the PID controller, which will be explained in more detail later. The input, in this case, will be the error associated with the light differences, and the output will be the angle of the motor. Using MATLAB and its System Identification Toolbox, the input and output data are then transformed into a ".mat" file, and the transfer function is derived utilizing MATLAB's transfer function estimation. The system's transfer function is shown in Equation 2.

$$\text{Transfer Function} = \frac{0.3246s + 0.08954}{s + 0.344} \quad (2)$$

Secondly, once the transfer function is derived, a verification process is performed to ensure that the transfer function is correctly modeled after the measured data. To do this, a set of error data is gathered and plotted with respect to time, and then the function of this graph is formulated, i.e., $f(x)$, where x is time and $f(x)$ is the value based on time. This function will then be used as an input for Simulink, which will simulate a range of input errors responsible for the servo's angle. Finally, the input is connected to the derived transfer function and tested.

Once the transfer function is verified, the next step is to derive the discrete transfer function for the PID controller. Using MATLAB's function "c2d", the continuous transfer function is then transformed into a discrete transfer function as seen in Equation 3. Similarly, verify the transfer function using the same method; in this case, the discrete transfer function.

$$\text{Discrete Transfer Function} = \frac{0.3246z - 0.3237}{z - 0.9966} \quad (3)$$

Finally, once everything has been verified, a PID control system implementing an input wave from the measured error values and the transfer function, a PID controller can be made to simulate the real system. This has the purpose of showing the user a visual representation of how the input controls the angle of the motor, and how PID control will allow the system to operate without any oscillations or glitches.

IV. CONTROLLER DESIGN DETAILS

After constructing the system, the following criteria will be met: the platform will rotate to follow the brightest light source in the room, the controller will accurately identify the optimal position, and the system will have a final error of 10% or less. We will be using a PID, or Proportional-Integral-Derivative, controller, which will accurately set the angular position of the motor using the K_p , K_i , and K_d values as well as the motor's current position. The PID controller was implemented in this system and is shown in Figure 3, which utilizes the system gain as an input for the servo motor, and feedback is determined through the photoresistors.

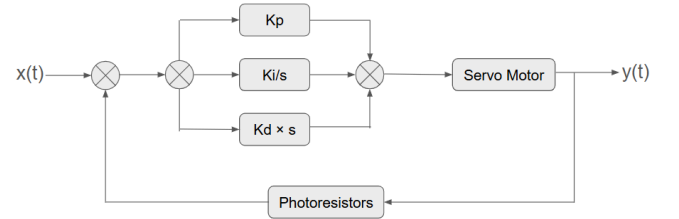


Figure 3: PID Control System Block Diagram

V. CONTROLLER IMPLEMENTATION DETAILS

First, the transfer function is determined by utilizing how much voltage is being transferred into the motor's PWM control. The completed MATLAB simulation file is demonstrated through Figure 4. This simulation allows for the testing of the input from the photoresistors through the transfer function and the output on the scope.

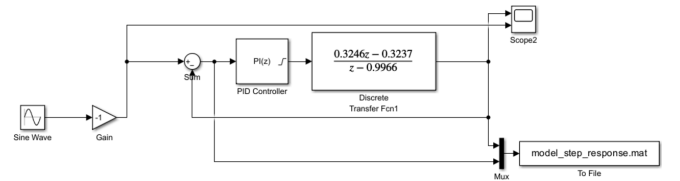


Figure 4: MATLAB Simulation

A sine wave was initialized to represent the difference between the photoresistors. The information was input to the PID controller and the discrete transfer function. The results were then displayed on a scope.

Figure 5 displays the input and output measurements of the system. The output changes according to the input, as seen by the input and output waveforms. In other words, the system is fast in its response. The system's output changes quickly based on the system's input.

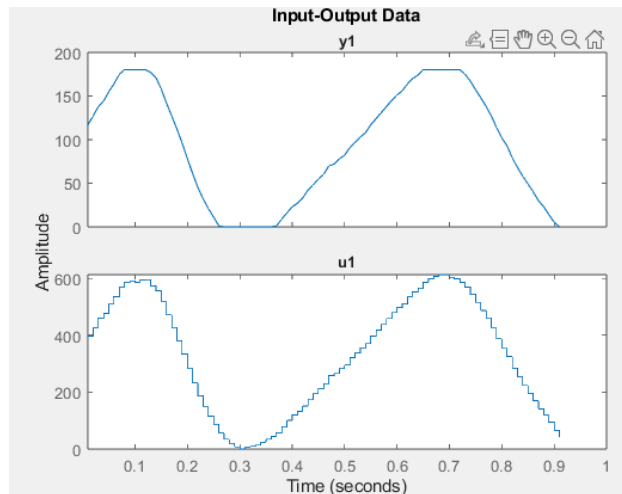


Figure 5: Input vs Output measurements

The PID (Proportional Integral Derivative) controller is supposed to maintain the output such that the difference between the measurements on the photoresistors is equal to zero. Using PID tuning, we can find the values of the proportional, integral, and derivative gains (also known as k_p , k_i , and k_d). Figure 6 displays the output of the system after determining the values of k_p , k_i , and k_d . As seen in Figure 6, the system tries to maintain the desired output. However, it keeps fluctuating around the desired output with a small margin of error.

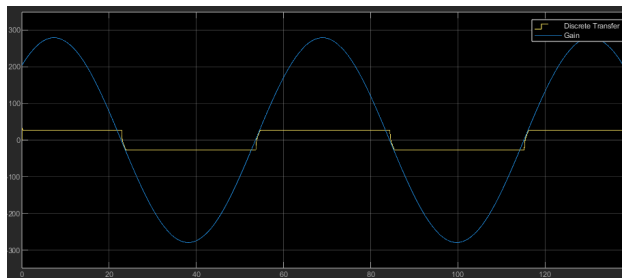


Figure 6: PID tuning

For the programming of the microcontroller, the resistance values of the photoresistors were read, the PID controller was established, the difference between the two photoresistors was calculated, and the direction of the servo motor was determined. As seen in Figure 7, the entirety of the program is looped for constant readings and movement adjustments. The resistance values were read from the “analogRead()” command, and the difference, or error, between the right and left sensors was found. Next, the PID of the system was determined, which utilized K_p , K_i , and K_d values of 1, 0.5, and 0.01, respectively.

```
void loop() {
    //Defining Time Values
    current_time = micros();
    time_change = (current_time - previous_time) / 1e6;
    previous_time = current_time;

    //Reading Values From Photoresistors
    right_sensor = analogRead(D2);
    left_sensor = analogRead(D3);

    //Math for Input And Error
    input = abs((right_sensor + left_sensor) / 1000);
    error = goal - input;

    //Math For PID Controller Components
    P = kp * (error - error_1);
    I = ki * time_change * error_1;
    D = (kd / time_change) * (error_2 * error_1 + error_2);

    //Math For PID Output
    output = output_1 + P + I + D;
    if(abs(error) < 0.5){
        output = 0;
    }

    //Redefining Errors
    error_2 = error_1;
    error_1 = error;
    output_1 = output;
}
```

Figure 7: Reading Photoresistor and Calculating PID

For the motor direction, conditional statements were implemented to determine which photoresistor sensed more light. A photoresistor’s resistance and the light intensity are inversely related, so if the right sensor’s resistance was greater than the left sensor’s resistance, that would indicate a stronger light intensity on the left side, and the motor would be instructed to move towards the left. Additionally, since a servo motor was utilized, its maximum range of motion was limited to between 0° and 180° . This is demonstrated through Figure 8.

```
//Motor Controls From PID Output
//Motor Direction
if(output != 0) {
    if((right_sensor - left_sensor) > 0){
        direction = 1;
    }
    else{
        direction = -1;
    }
    //Motor Movement
    position = position_1 + int(direction * output);
    if(position > 180){
        position = 180;
    }
    if(position < 0){
        position = 0;
    }
    position_1 = position;
}
else {
    position = position_1;
}
```

Figure 8: Motor Control from PID Output

VI. CONTROLLER EVALUATION

Figure 9 shows the final response for the system compared to the predicted response. It demonstrates that the actual motor reflects our simulation with 94.68% accuracy. This result satisfies our goal of creating a system with less than 10% error. Figure 10 shows the result of the discrete-time transfer function (in the z-domain) compared to the measured result. Similarly, Figure 11 shows the same concept but with the continuous transfer function in the s-domain.

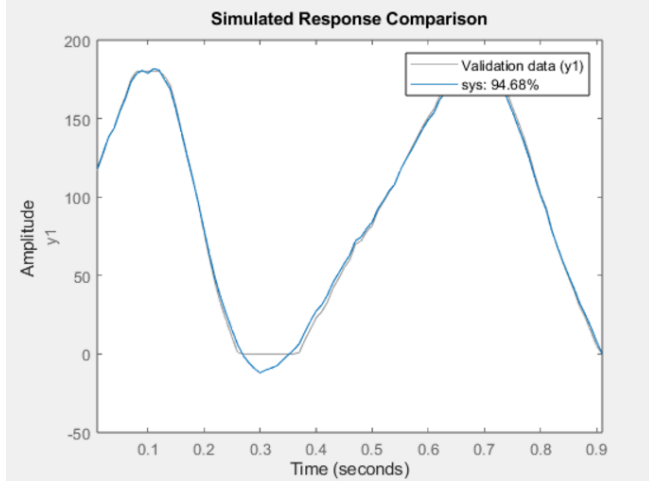


Figure 9: Simulated Response Comparison

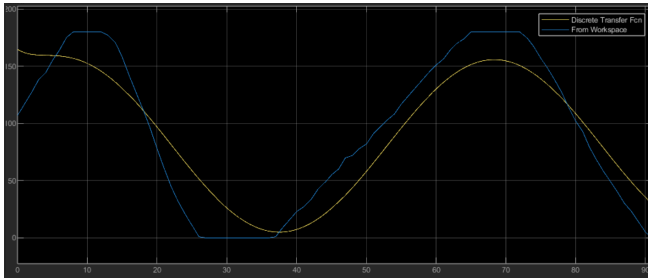


Figure 10: Discrete-Time Transfer Function Comparison

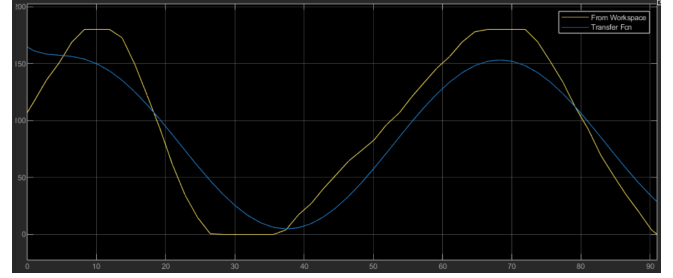


Figure 11: Continuous-Time Transfer Function Comparison

VII. DISCUSSION

In summary, the prototype solar panel system did work somewhat as expected. The system did respond to the difference between the readings of the photoresistors. In other words, the motor moved when the photoresistors gave different readings. However, the system was also very unstable. To add to this statement, the motor would sometimes make the solar panel move back and forth even when the light source was held in a constant position.

REFERENCES

- [1] "What's the best angle for solar panels to get maximum output?" Sunrun, <https://www.sunrun.com/knowledge-center/best-angle-for-solar-panels> (accessed Nov. 15, 2024).
- [2] "Array Skylink," ARRAY Technologies, https://arraytechinc.com/products/array-skylink/?gad_source=1&gclid=Cj0KCQiA_9u5BhCUARIsABbMSPvrkUP5ecxYK4O14yFctwjM1fexYdPsvCvEROZvhBJyWYfmzlu839saAu2hEALw_wcB (accessed Nov. 15, 2024).